Protocoles de l'IOT

CoAP, MQTT

CoAP
Constrained Application Protocol

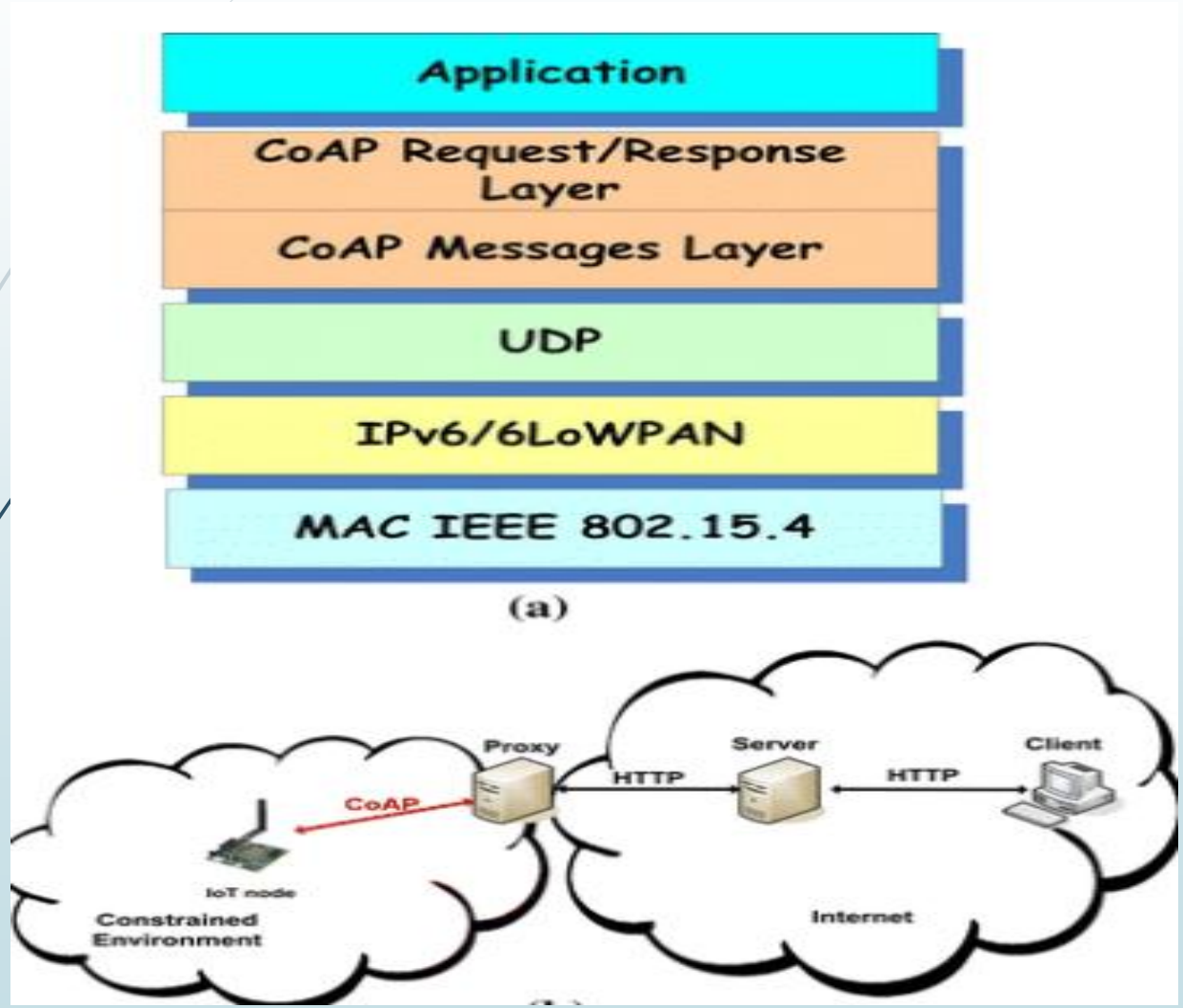
- Protocole de communication pour l'Internet des objets (IoT)
- Conçu pour les objets et réseaux contraints
- Standardisé par l'IETF
- Modèle de communication proche du Web/REST

- Définition
- **Le protocole CoAP (Constrained Application Protocol) est un protocole de type requête/réponse, basé comme HTTP sur l'architecture REST. Le protocole est optimisé, comme son nom l'indique, pour les périphériques et les réseaux à ressources limitées.**

Pourquoi CoAP et pas HTTP?

- Les objets IoT présentent souvent des contraintes :
  - Mémoire limitée
  - Faible puissance de calcul
  - Batterie limitée
  - Débit réseau réduit
  - Réseaux avec pertes de paquets
  - Besoin de communications simples et peu coûteuses
- **CoAP répond à ces contraintes avec un protocole léger**
- Le protocole HTTP
  - n'est pas approprié aux dispositifs à ressources limitées
  - Il est basé sur la couche de transport TCP, qui est connu pour un comportement problématique sur les réseaux sans fil avec pertes
  - Il ne réalise pas certains mécanismes importants pour les scénarios du Web of Things, comme la communication bi-directionnelle ou la découverte des ressources ou des dispositifs

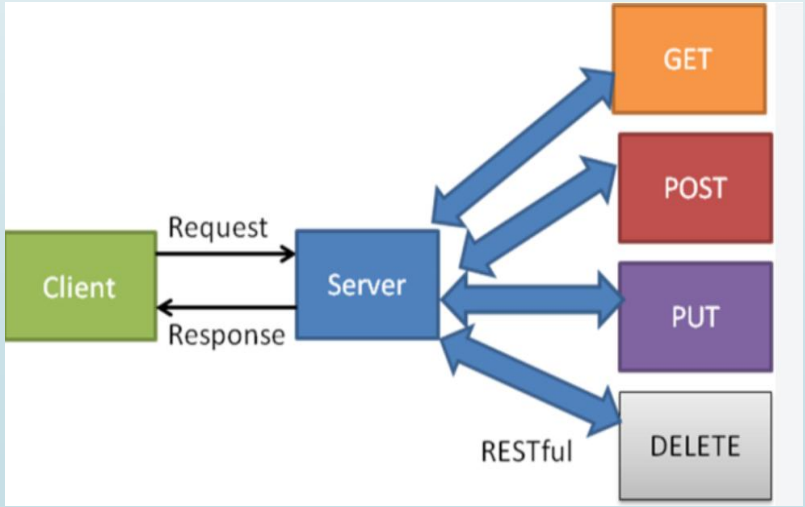
Architecture de CoAP



- 6LoWPAN signifie IPv6 over Low-Power Wireless Personal Area Networks.
- IPv6 sur des réseaux personnels sans fil à faible consommation d'énergie.
- IEEE 802.15.4 est une norme qui définit les caractéristiques des communications sans fil à faible débit.
- Les trames IEEE 802.15.4 sont petites : leur taille maximale est de 127 octets au niveau MAC.

Les méthodes CoAP

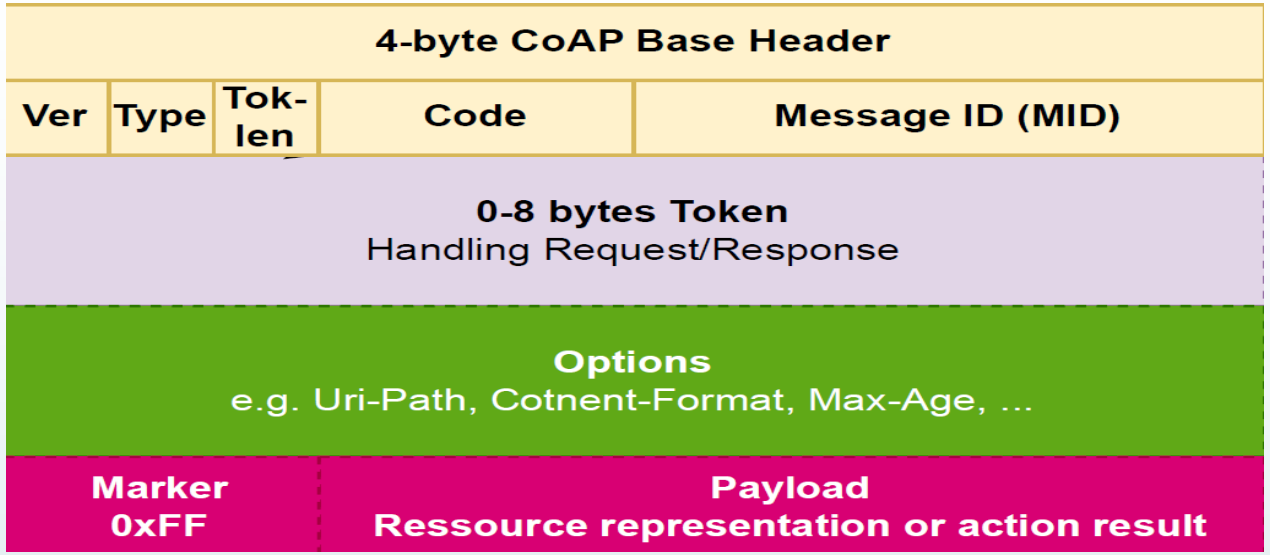
Méthode	Fonction
GET	Lire une ressource
POST	Créer une ressource ou déclencher une action
PUT	Créer ou modifier une ressource
DELETE	Supprimer une ressource



GET coap://sensor01/temperature

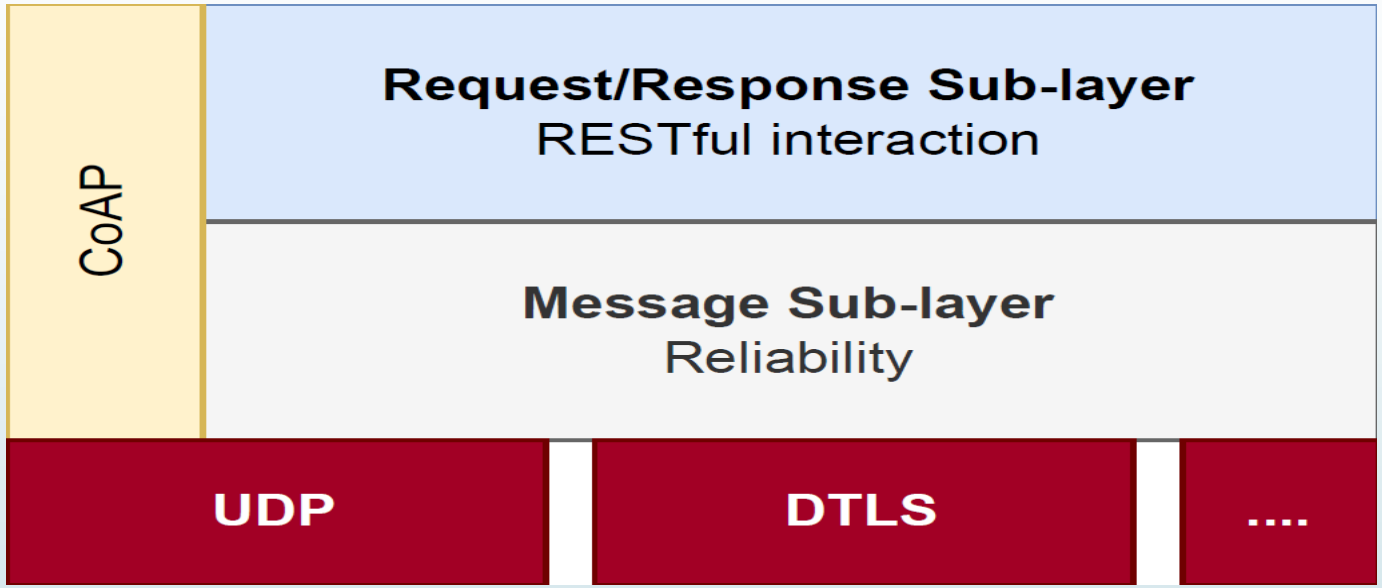
demande la valeur actuelle de la température.

Structure d'un message CoAP



- Un message CoAP contient principalement :
- **Header** : possède seulement **4 octets**.
  - Cet entête contient:
  - la version (2bits),
  - le type (2 bits): CON, NON, ACK ou RST
  - la taille du token (4 bits),
  - le code sur 1 octet : 0.01 GET, 0.02 POST, 2.05 Content, 4.04 Not Found…
  - le Message ID sur 2 octets pour la détection des doublons ; Token : association requête / réponse
  - **Token** : Le token est utilisé par la sous-couche Request/Response afin d'associer les réponses aux requêtes correspondantes
- **Options** : peut contenir de 0 à n options. codage compact en delta, équivalent des en-têtes HTTP. Cette section se termine avec un marker 0xFF.

Les sous couches de CoAP

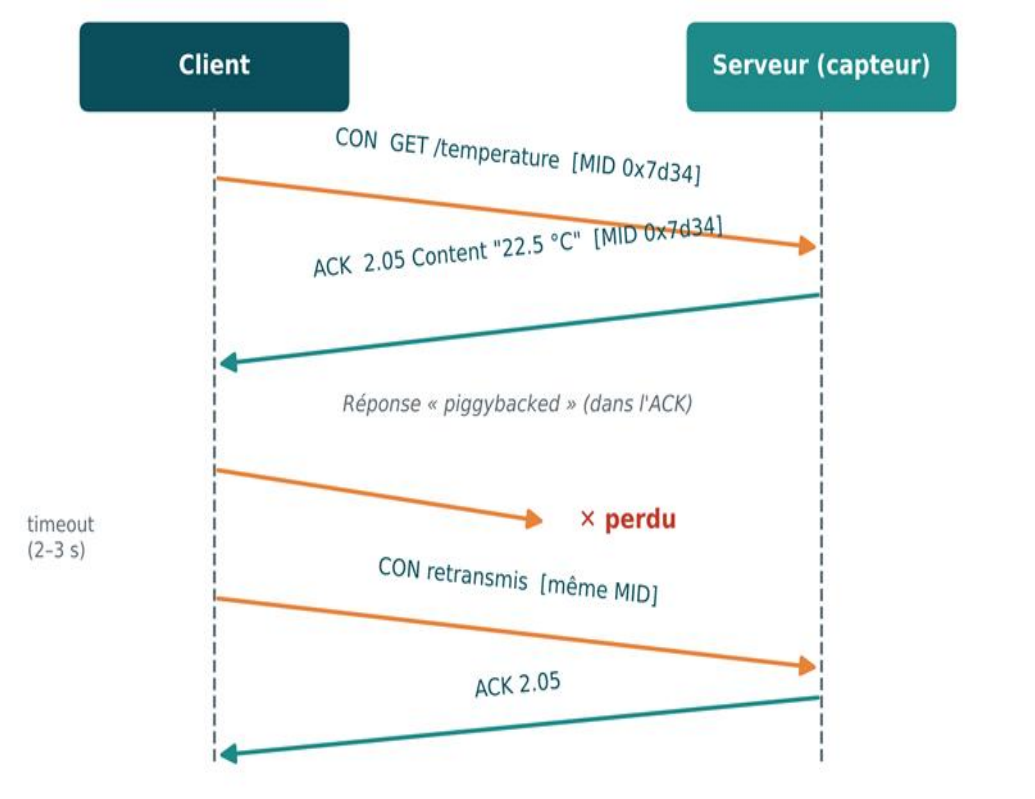


- La sous-couche Request/Response a pour mission de gérer les messages CoAP suivant le modèle requête-réponse.
- La sous-couche Message a pour mission de gérer la qualité de service (retransmission et déduplication des messages).

La sous-couche Message
Types des messages CoAP

- **La sous-couche Message est responsable de la retransmission et de la déduplication des messages. Pour ce faire, le standard définit 4 types de messages différents :**
- **CON — Confirmable**
  - Message nécessitant une confirmation
  - Permet une communication fiable
- **NON — Non-confirmable**
  - Pas de confirmation demandée
  - Adapté aux données périodiques
- **ACK — Acknowledgement**
  - Accusé de réception d'un message CON
- **RST — Reset**
  - Indique qu'un message ne peut pas être traité

Communication CON/ACK



- Un message CON est retransmis tant qu'aucun ACK ne revient
- Délai initial aléatoire entre 2 et 3 s (ACK_TIMEOUT = 2 s, facteur 1,5), doublé à chaque tentative
- Au plus 4 retransmissions (MAX_RETRANSMIT)
- Le Message ID permet d'éliminer les doublons

La sous-couche Request/Response

- Les messages CoAP contiennent tous un code, qui représente une méthode pour une requête et un statut pour une réponse.

Les codes de réponse

- Les codes sont organisés en classes :
  - **2.xx** → succès
  - **4.xx** → erreur dans la requête
  - **5.xx** → erreur du serveur

Code	Signification
2.01	Created
2.02	Deleted
2.04	Changed
2.05	Content
4.04	Not Found
5.00	Internal Server Error

Les options CoAP

- Les options permettent de compléter une requête ou une réponse.
- Exemples :
  - Uri-Path
  - Uri-Query
  - Content-Format
  - Accept
  - Observe
  - Block1
  - Block2
- Exemple :
  - GET /temperature?unit=celsius

CoAP Observe

- Le mécanisme Observe permet à un client de surveiller une ressource.
- Sans Observe :
  - Client —— GET ——> Capteur
  - Client —— GET ——> Capteur
  - Client —— GET ——> Capteur ...
- Avec Observe :
  - Client —— Observe ——> Capteur
  - Capteur —— Notification ——> Client
  - Capteur —— Notification ——> Client
- Très utile pour :
  - température
  - humidité
  - Luminosité
  - état d'un équipement



Découverte des ressources

- CoAP permet de découvrir les ressources proposées par un objet.
- Ressource :
  - /.well-known/core
- Exemple conceptuel :
  - </temperature>
  - </humidity>
  - <battery>
  - <light>
- Cela facilite l'intégration automatique des objets IoT.

CoAP et sécurité

- CoAP peut utiliser :
- DTLS (Datagram Transport Layer Security) est un protocole de sécurité qui permet de protéger les communications CoAP lorsqu'elles utilisent UDP.
  - CoAP - DTLS - UDP
- DTLS permet notamment :
  - Confidentialité
  - Intégrité
  - Authentification
- CoAP peut également être sécurisé avec OSCORE, qui protège les messages CoAP au niveau applicatif.

CoAP vs HTTP

Critère	CoAP	HTTP
Domaine	IoT	Web
Transport typique	UDP	TCP / QUIC
Overhead	Faible	Plus important
Modèle	REST	REST
Ressources limitées	Oui	Moins adapté
CON/ACK	Oui	Non
Observe	Oui	Pas directement

Exemple d'application IoT

- **Système intelligent de surveillance d'une maison**
- Un capteur expose :
  - /temperature
  - /humidity
  - /light
  - /motion
  - /battery
- Exemples :
  - GET /temperature
  - GET /humidity
  - GET /battery
  - PUT /light
  - Observe /motion
- **CoAP permet ainsi à une passerelle ou à une application de communiquer avec les objets.**

Avantages et limites

- **Avantages**
  - Léger
  - Faible overhead
  - Adapté aux objets contraints
  - REST
  - Fonctionne avec UDP
  - Observe
  - Découverte des ressources
  - Mécanismes de fiabilité
- **Limites**
  - UDP peut être soumis aux pertes de paquets
  - Sécurité à configurer correctement
  - Moins adapté que MQTT pour certains scénarios de diffusion événementielle
  - Interopérabilité avec le Web nécessite parfois une passerelle/proxy

Questions de compréhension

- Pourquoi CoAP utilise-t-il généralement UDP ?
- Quelle est la différence entre CON et NON ?
- Quel est le rôle de ACK ?
- Quelle est la différence entre CoAP et HTTP ?
- À quoi sert Observe ?
- Quel est le rôle de /.well-known/core ?
- Quelle différence fondamentale existe entre CoAP et MQTT ?
- Donner un exemple d'application IoT utilisant CoAP.

Message Queuing Telemetry Transport - MQTT

MQTT — Message Queuing Telemetry Transport

- Protocole léger de messagerie pour l’IoT
- Communication publish/subscribe
- Fiable, simple et adapté aux réseaux contraints

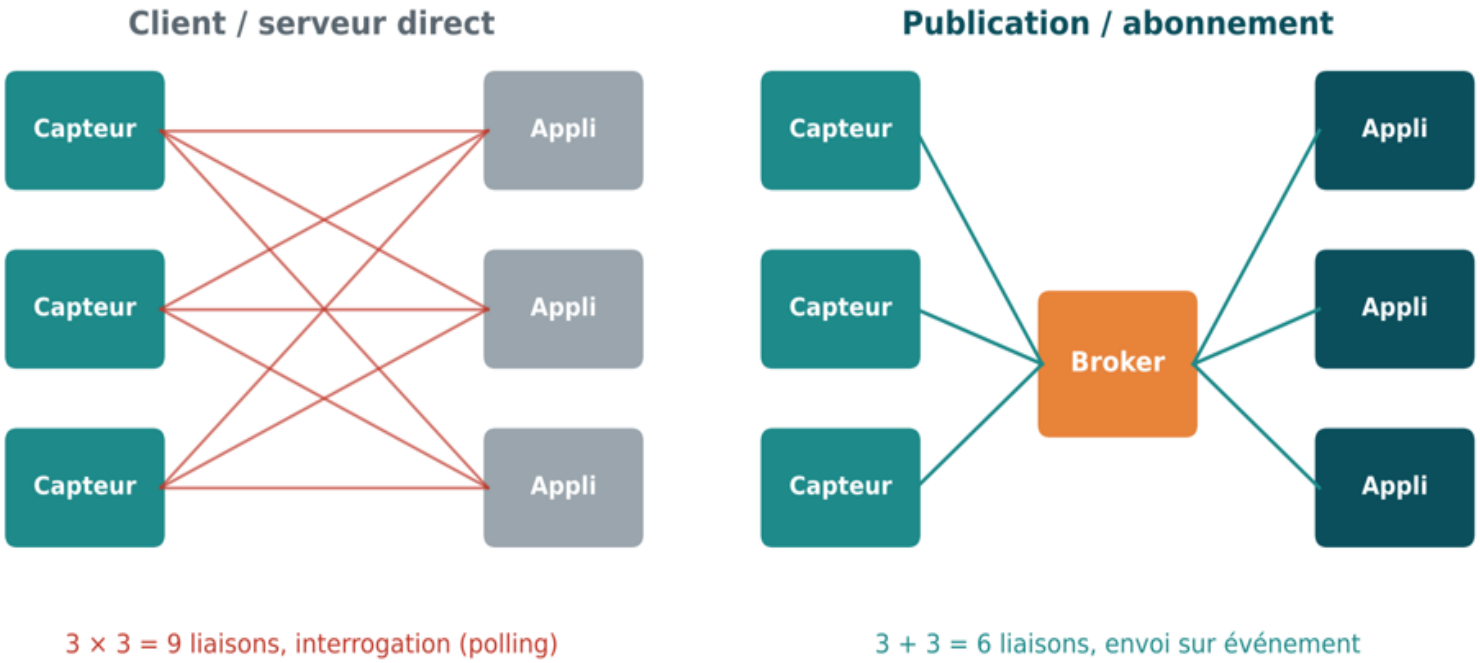
Objectifs de MQTT

- Échanger des données avec une faible surcharge réseau
- Découpler les producteurs et consommateurs de données
- Fonctionner sur des réseaux à faible débit ou instables
- Faciliter la communication entre objets connectés et applications

Pourquoi MQTT pour l'IoT ?

- Messages courts et faible overhead (surcharge): MQTT ajoute très peu de données supplémentaires à chaque message envoyé
- Communication asynchrone: l'émetteur et le récepteur n'ont pas besoin d'être disponibles au même moment pour échanger des messages.
- Nombreux clients connectés à un même broker
- Prise en charge de différents niveaux de qualité de service (QoS): niveau de garantie choisi pour la livraison d'un message entre un client MQTT et le broker, ou entre le broker et un abonné
- Possibilité de conserver certains messages avec le mécanisme Retained Message (message conservé): garder en mémoire le dernier message publié sur un topic, afin de le transmettre immédiatement aux nouveaux abonnés qui s'abonnent à ce topic.

Problématique : relier de nombreux objets



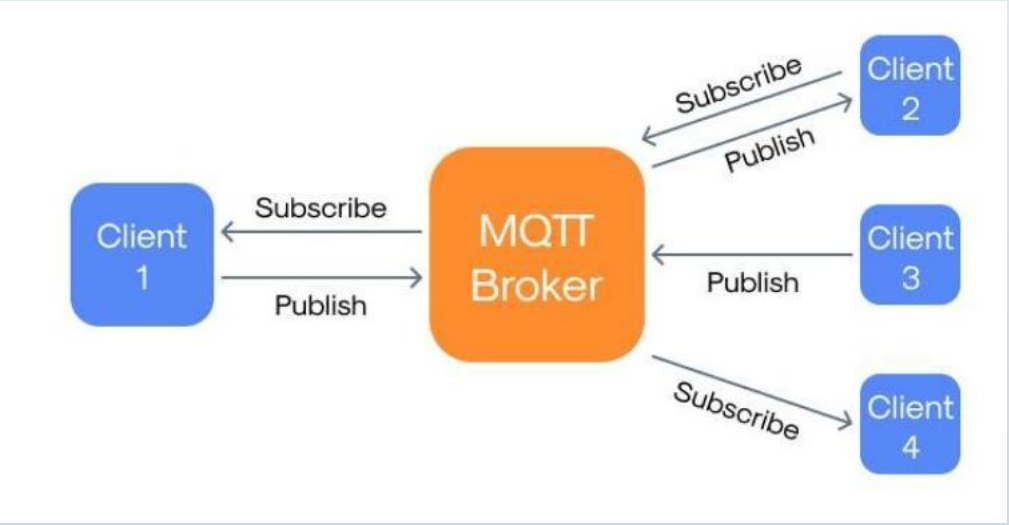
Comment faire circuler les données de milliers de capteurs vers de nombreuses applications, en temps réel, sur des réseaux peu fiables et avec des objets à faibles ressources ?

Les éléments fondamentaux

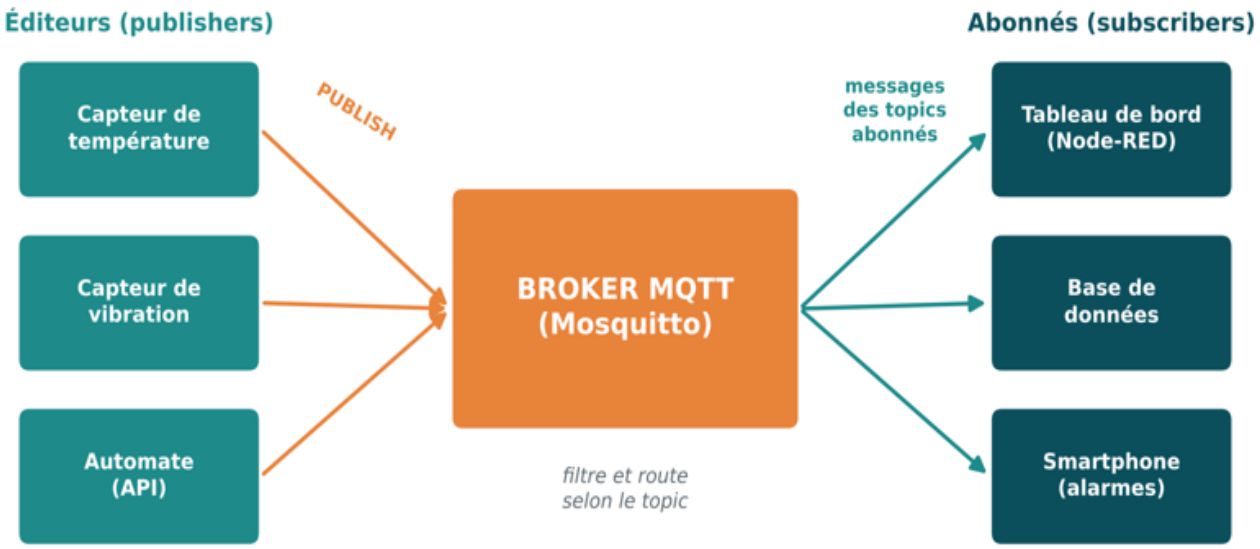
- Client MQTT : capteur, actionneur, application mobile, serveur…
- Broker : serveur MQTT qui reçoit et distribue les messages
- Topic : nom logique utilisé pour organiser les messages
- Message : charge utile (payload) publiée sur un topic
- Subscription : abonnement d’un client à un ou plusieurs topics

Le modèle Publish / Subscribe

- 1. Un client publie un message sur un topic.
- 2. Le broker reçoit le message.
- 3. Le broker recherche les abonnés correspondants.
- 4. Le broker transmet le message aux subscribers.
- Avantage : publisher et subscriber n'ont pas besoin de se connaître.



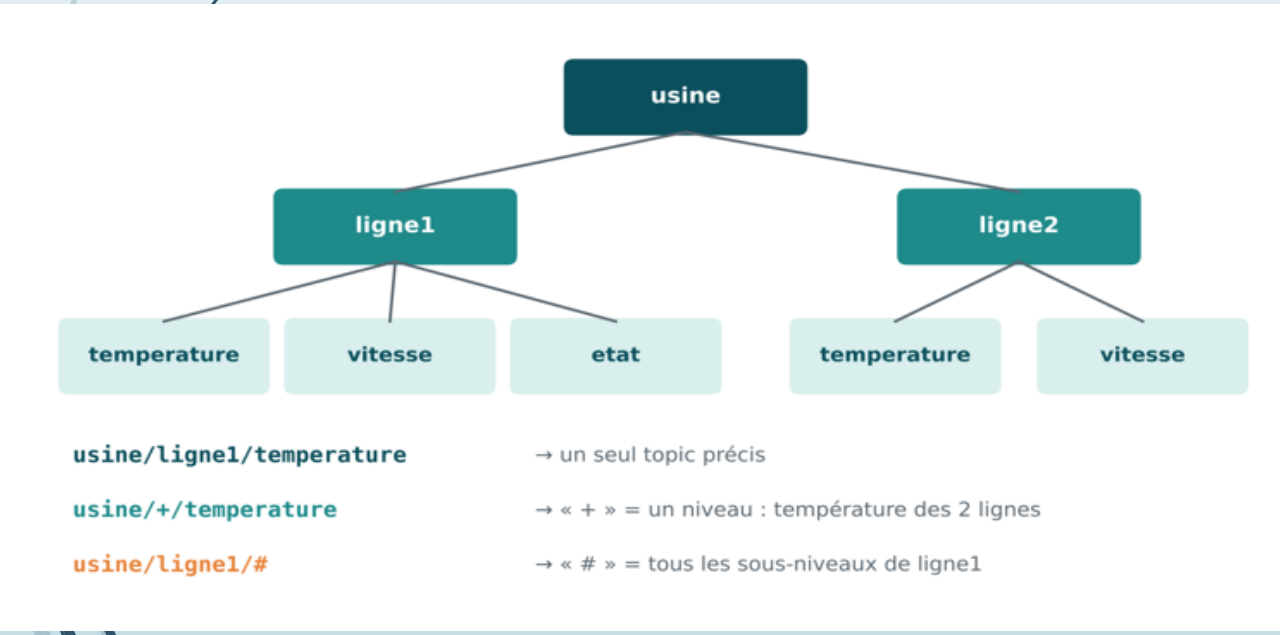
Architecture publication / abonnement



- **Découplage dans l'espace :** les clients ne connaissent pas leurs adresses
- **dans le temps :** l'abonné peut être absent lors de l'envoi
- **et en synchronisation :** l'éditeur n'est jamais bloqué

Les Topics MQTT

- Structure hiérarchique : maison/salon/temperature
- Exemple : ustomb/iot/salle01/temperature
- Le caractère + remplace un niveau : maison/+/temperature
- Le caractère # représente plusieurs niveaux : maison/#
- Les topics permettent de structurer et filtrer les flux de données.



- Chaîne UTF-8 hiérarchique, niveaux séparés par « / »
- Sensible à la casse : Usine ≠ usine
- « + » remplace un niveau, « # » tous les niveaux suivants (en dernière position)
- Les jokers sont réservés aux abonnements, jamais à la publication
- Les topics $SYS/… donnent l'état du broker

Exemple de communication

- Capteur → publish : topic = maison/salon/temperature
- Payload = 24.7 °C
- Application → subscribe : maison/+/temperature
- Le broker transmet automatiquement la mesure à l’application.
- Plusieurs applications peuvent s’abonner au même topic.

Paquet MQTT

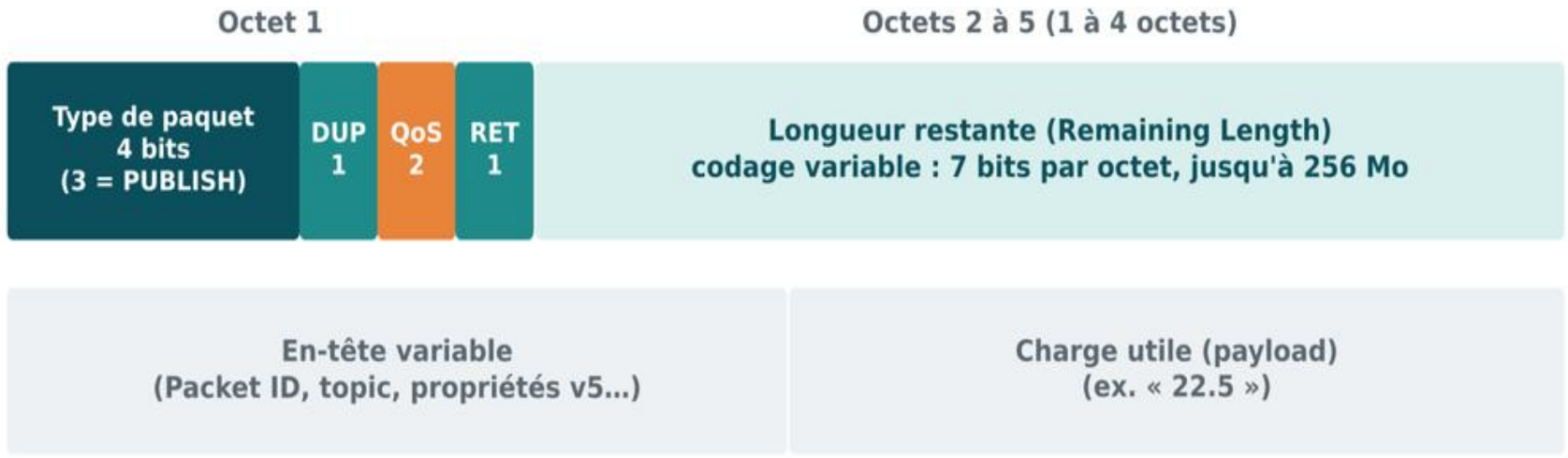
tous les paquets MQTT n'ont pas exactement les mêmes champs. Leur structure dépend du type de paquet :

- publication d'un message (PUBLISH),
- abonnement (SUBSCRIBE),
- connexion (CONNECT),
- accusé de réception (PUBACK), etc.

Le protocole MQTT organise généralement les paquets autour de trois parties :

1. Fixed Header (en-tête fixe) : présent dans tous les paquets MQTT.
2. Variable Header (en-tête variable) : présent uniquement lorsque le type de paquet le nécessite.
3. Payload (charge utile) : contient les données utiles ou les informations supplémentaires, selon le type de paquet. Cette partie est facultative pour certains paquets.

Format d'un paquet MQTT



En-tête fixe minimal : 2 octets (ex. PINGREQ = 0xC0 0x00)

1 CONNECT	2 CONNACK	**3 PUBLISH**	4 PUBACK	5 PUBREC
6 PUBREL	7 PUBCOMP	8 SUBSCRIBE	9 SUBACK	10 UNSUBSCRIBE
11 UNSUBACK	12 PINGREQ	13 PINGRESP	14 DISCONNECT	15 AUTH (v5)

Type de paquet

Valeur	Type	Rôle
1	CONNECT	Demander une connexion
2	CONNACK	Confirmer la connexion
3	PUBLISH	Publier un message
4	PUBACK	Accusé de réception QoS 1
8	SUBSCRIBE	S'abonner à un topic
9	SUBACK	Confirmer l'abonnement
12	PINGREQ	Demander une réponse de maintien de connexion
13	PINGRESP	Répondre à PINGREQ

Flags — Indicateurs

DUP

Duplicate delivery

Indique si le paquet PUBLISH est une retransmission possible.

- `0` : première tentative d'envoi.
- `1` : retransmission du paquet.

QoS

Quality of Service

Les 2 bits indiquent le niveau de livraison : `00` pour QoS 0, `01` pour QoS 1 et `10` pour QoS 2.

La valeur `11` est interdite.

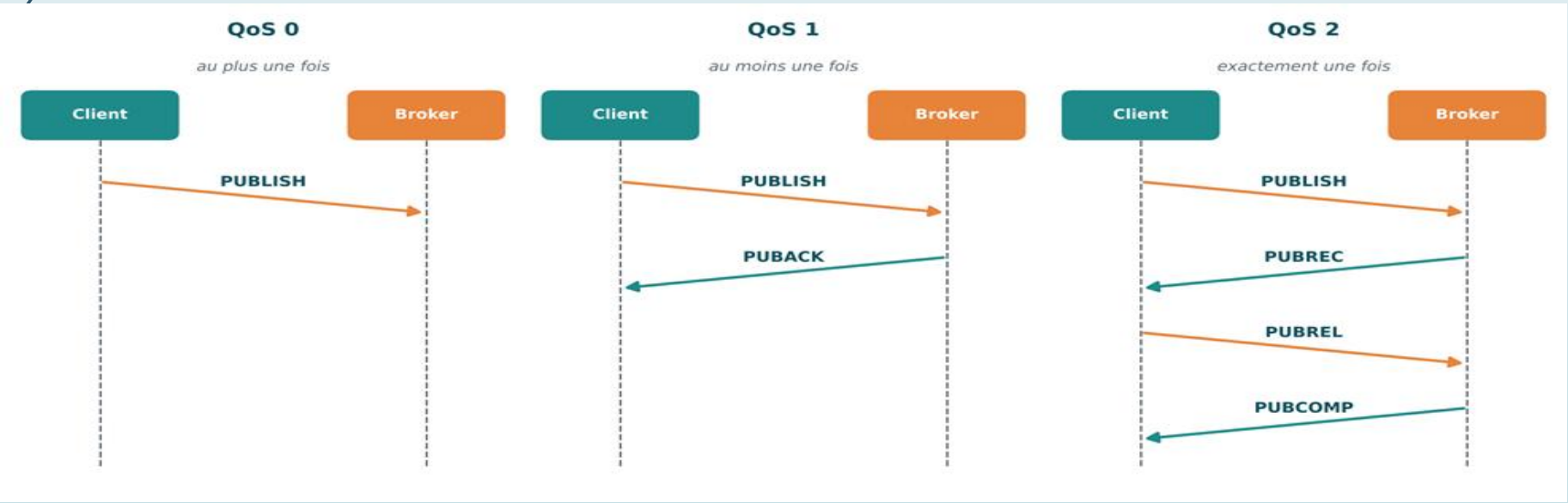
RETAIN

Message conservé

- `0` : la publication n'est pas demandée comme message retenu.
- `1` : le broker conserve ce message comme dernier message retenu du topic.

QoS — Quality of Service

- QoS 0 — At most once : message envoyé sans confirmation.
- QoS 1 — At least once : livraison garantie, mais doublons possibles.
- QoS 2 — Exactly once : livraison exactement une fois, avec davantage d'échanges.
- Le niveau de QoS est choisi selon les exigences de l'application.



QoS 0, 1 et 2 — Comparaison

- QoS 0 : faible surcharge, adapté aux mesures périodiques où une perte est acceptable.
- QoS 1 : compromis entre fiabilité et coût réseau.
- QoS 2 : fiabilité maximale, mais protocole plus lourd.
- Le choix dépend du débit, de la fiabilité requise et du coût des retransmissions.

Retained Message (Message retenu)

- Le broker peut conserver le dernier message publié sur un topic.
- Lorsqu’un nouveau client s’abonne, il peut recevoir immédiatement cette dernière valeur.
- Exemple : maison/salon/temperature → 24.7 °C
- Utile pour connaître rapidement l’état courant d’un équipement.

Remaining Length — Longueur restante

- Ce champ indique le nombre total d'octets qui suivent ce champ, c'est-à-dire la taille de l'en-tête variable et du payload réunis.
- Il occupe de 1 à 4 octets selon la taille du paquet.
- Exemple : si l'en-tête variable contient 24 octets et le payload 10 octets, alors :
- $$ \text{Remaining Length}=24+10=34 $$
- La valeur est donc 34, codée sur un octet dans cet exemple.

En-tête variable (Variable Header)

- Son contenu varie selon le type de paquet. Pour un paquet PUBLISH, il contient principalement les champs suivants :

Champ	Signification
Topic Name	Nom du topic sur lequel le message est publié, par exemple maison/temperature.
Packet Identifier	Identifiant du paquet, présent uniquement lorsque le QoS est 1 ou 2. Il sert à associer les paquets aux échanges de livraison correspondants.

- Le nom du topic est encodé comme une chaîne précédée de sa longueur sur 2 octets. L'identifiant de paquet, lorsqu'il est présent, occupe 2 octets.

Payload — Charge utile

- Le payload contient les données utiles du message.
- Par exemple, un capteur peut publier :
  - 25 : une température ;
  - {"temperature":25,"humidity":60} : des données au format JSON ;
  - une chaîne de caractères ou des données binaires.
- Le payload d'un paquet PUBLISH n'a pas de structure imposée par MQTT : son contenu dépend de l'application.

Exemple concret : un paquet PUBLISH

Un capteur publie la température `23.5` sur le topic `maison/salon/temperature`.

Dans MQTT 3.1.1, en supposant un message QoS 0, sans indicateur RETAIN :

Structure du paquet

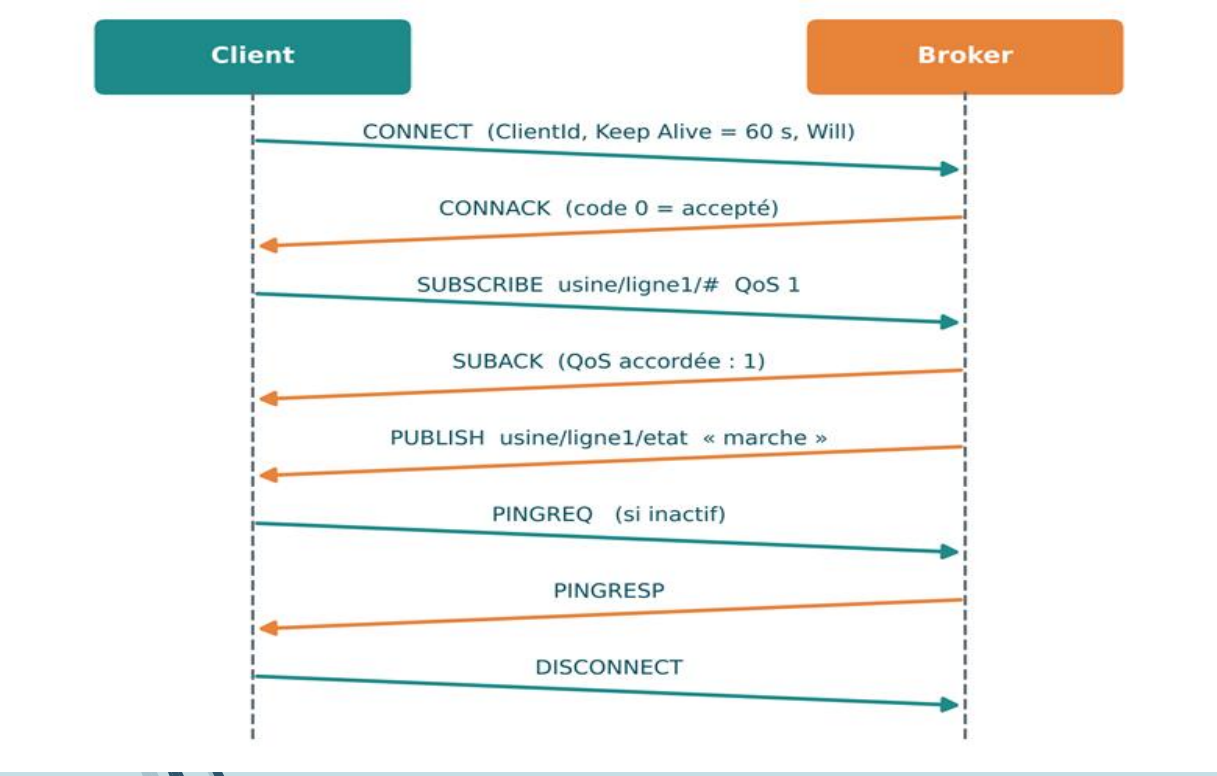
`0x30`	En-tête fixe : PUBLISH, QoS 0, RETAIN = 0
`0x1F`	Remaining Length = 31 octets
Topic	Longueur du topic + `maison/salon/temperature`
Payload	`23.5`

Connexion MQTT

- MQTT fonctionne généralement au-dessus de TCP/IP.
- Port TCP 1883 : MQTT sans chiffrement.
- Port TCP 8883 : MQTT sécurisé avec TLS.
- Les clients établissent une connexion avec le broker avant de publier ou s’abonner.

Cycle de vie d'une communication MQTT

- CONNECT → connexion du client au broker
- CONNACK → confirmation de connexion
- SUBSCRIBE / SUBACK → abonnement à un topic
- PUBLISH → publication d'un message
- UNSUBSCRIBE / UNSUBACK → suppression d'un abonnement
- DISCONNECT → fermeture normale de la connexion



- Le client ouvre une connexion TCP puis envoie CONNECT ; le broker répond par CONNACK
- SUBSCRIBE / SUBACK enregistrent les filtres de topics
- Keep Alive : sans échange pendant 1,5 × l'intervalle, le broker considère le client déconnecté : verifier que la connexion est toujours active avec pingreq et pingresp
- DISCONNECT ferme proprement la session

MQTT 3.1.1 et MQTT 5.0

- MQTT 3.1.1 : version largement déployée et simple à implémenter.
- MQTT 5.0 : ajoute davantage de contrôle et de métadonnées.
- Exemples MQTT 5.0 : propriétés, raisons de réponse, expiration des messages.
- Le choix dépend des équipements, brokers et bibliothèques disponibles.

Exemple d’application IoT

- Capteur de température → publish → maison/salon/temperature
- Broker MQTT → distribue la mesure
- Tableau de bord → subscribe → affiche la température
- Application d’alerte → subscribe → déclenche une notification si nécessaire
- Plusieurs consommateurs peuvent exploiter le même flux.

Avantages et limites

- Avantages : léger, simple, asynchrone, scalable, adapté à l'IoT.
- Avantages : QoS, retained messages, LWT et architecture découplée.
- Limites : dépendance au broker pour l'architecture classique.
- Limites : TCP ajoute une surcharge par rapport à certains protocoles très contraints.
- Limites : la sécurité doit être correctement configurée.

MQTT vs HTTP — aperçu

- MQTT : publish/subscribe, communication événementielle, messages légers.
- HTTP : requête/réponse, modèle client/serveur, très répandu sur le Web.
- MQTT est particulièrement adapté aux flux IoT continus et asynchrones.
- HTTP reste très pratique pour les API Web et les échanges requête/réponse.

MQTT vs CoAP — aperçu

- MQTT : publish/subscribe, généralement sur TCP, broker central.
- CoAP : modèle REST, généralement sur UDP, adapté aux dispositifs contraints.
- MQTT est naturel pour la diffusion de données et la télémétrie.
- CoAP est naturel pour l’accès aux ressources d’un objet.

Quand utiliser CoAP et MQTT ?

CoAP

Communication de type requête-réponse

On privilégie CoAP lorsqu'un équipement doit **interroger un autre équipement ou agir directement sur lui**, avec peu de surcharge protocolaire.

Exemples :

- Lire la température d'un capteur.
- Allumer ou éteindre une lampe connectée.
- Consulter l'état d'un actionneur sur un réseau local contraint.

Fonctionne généralement sur UDP. Peut être sécurisé avec DTLS.

MQTT

Publication-abonnement

On privilégie MQTT lorsqu'il faut **diffuser des données à plusieurs applications** en passant par un broker, sans que les capteurs aient à connaître directement les destinataires.

Exemples :

- Transmettre périodiquement les températures de centaines de capteurs.
- Superviser une usine en temps réel.
- Envoyer des mesures à plusieurs tableaux de bord.

Fonctionne généralement sur TCP. Peut être sécurisé avec TLS.

Résumé

- MQTT est un protocole de messagerie léger conçu pour les environnements connectés.
- Son architecture repose sur Publisher, Broker, Subscriber et Topics.
- Les QoS 0/1/2 permettent d’adapter la fiabilité aux besoins.
- Retained Messages apportent des mécanismes utiles pour l’IoT.
- TLS, authentification et contrôle d’accès sont essentiels pour sécuriser MQTT.